Trabalho 1 - Sistemas Operacionais

Disciplina: Sistemas Operacionais

Professor: Luiz Fernando Bessa Seibel

Aluno: Cristiano Ferreira de Mattos - 2320397

Aluno: Pedro Mesquita Maia - 2312664

Data: 19 de Outubro de 2026



1. Objetivo

O objetivo do trabalho é programar em C, utilizando as primitivas fork/exec, sinais Unix e comunicação interprocessos (IPC), um simulador de núcleo de sistema operacional (Kernel Sim).

O kernel deve gerenciar de 3 a 6 processos de aplicação (A1 a A6), escalonando suas execuções preemptivamente. O escalonamento é baseado na intercalação dos processos a partir da espera por operações de dispositivo de E/S (D1) ou do sinal de aviso indicando que a fatia de tempo acabou (time slice).

Para a emissão das interrupções baseadas no tempo e na E/S, é implementado um processo independente controlador de interrupções (InterController Sim).

2. Como foi dividido:

O trabalho foi estruturado em quatro arquivos, responsáveis pelos papéis de núcleo, controlador de interrupções e aplicações:

- kernel.c (Kernel Sim): É o processo principal, responsável por inicializar as estruturas de controle (Blocos de Controle de Processo – PCBs), alocar a memória compartilhada para armazenar o contexto (PC) e controlar duas filas: a fila de processos prontos (READY) e a fila FIFO de processos bloqueados aguardando o dispositivo de E/S (D1). O kernel é dirigido a interrupções e realiza as trocas de contexto suspendendo as aplicações com SIGSTOP e retomando-as com SIGCONT. Permanece em execução até receber SIGINT (Ctrl+C) ou SIGTERM. Ao encerrar, finaliza o InterController e as aplicações, recolhe os processos filhos com waitpid e remove o segmento de memória compartilhada.

- intercontroller.c (InterController Sim): Processo que executa paralelamente ao kernel e atua como controlador de relógio e de dispositivo. Emite a interrupção IRQ0 (SIGUSR1) a cada segundo, indicando o fim do time slice. Também recebe os avisos de E/S do kernel e processa os pedidos em série, aguardando 3 segundos por operação antes de emitir a interrupção IRQ1 (SIGUSR2), que indica o término da E/S.
- app.c (Processo de Aplicação): Processo filho criado inicialmente parado por meio de raise(SIGSTOP). Contém o laço principal da aplicação, executado até atingir MAX iterações, incrementando progressivamente o contador PC. Quando o valor do PC corresponde a um ponto configurado no roteiro, a aplicação envia um sinal ao kernel solicitando uma syscall de leitura (R) ou escrita (W) no dispositivo D1.
- common.h: Arquivo de cabeçalho que define as constantes de tempo, as estruturas compartilhadas em memória e o mapa de sinais utilizado na comunicação entre os processos:
  SIGUSR1: IRQ0, interrupção de fim de time slice.
  SIGUSR2: IRQ1, interrupção de conclusão de E/S.
  SIGRTMIN: syscall de leitura (R), enviada pela aplicação ao kernel.
  SIGRTMIN+1: syscall de escrita (W), enviada pela aplicação ao kernel.
  SIGRTMIN+2: aviso de solicitação de E/S enviado pelo kernel ao InterController.

3. Compilação e execução

```
gcc -Wall -Wextra -g -o kernel kernel.c
gcc -Wall -Wextra -g -o intercontroller intercontroller.c
gcc -Wall -Wextra -g -o app app.c
```

Os três executáveis devem ficar na mesma pasta, e o kernel deve ser executado a partir dela, porque ele inicia o InterController e as aplicações com exec("./intercontroller") e exec("./app"). Além disso, o programa exige de 3 a 6 argumentos, um para cada aplicação. Cada argumento define o número máximo de iterações (MAX) e, opcionalmente, um roteiro com as operações de leitura (R) e escrita (W).

Exemplo:

```
./kernel 8:3R,6W 5 5:2W
```

Esse comando inicializa:

- A1: 8 iterações, solicitando leitura (R) no PC 3 e escrita (W) no PC 6.
- A2: 5 iterações, sem operações de E/S.
- A3: 5 iterações, solicitando escrita (W) no PC 2.

O kernel permanece em execução mesmo após o término das aplicações. Para encerrar o simulador e liberar os recursos utilizados, pressione Ctrl+C.

Exemplo do laço principal do kernel

O kernel utiliza um laço dirigido a interrupções. Em vez de consultar continuamente o estado das aplicações, ele aguarda a chegada de sinais e executa os tratamentos correspondentes.

```
while (!fim_flag) {
    sigsuspend(&espera);

    if (chld_flag) {
        chld_flag = 0;
        tratar_sigchld();
    }

    for (int k = 0; k < npend; k++) {
        tratar_syscall(pend[k].pid, pend[k].op);
    }
    npend = 0;

    if (irq1_flag) {
```

```
        irq1_flag = 0;
        tratar_irq1();
    }

    if (irq0_flag) {
        irq0_flag = 0;
        tratar_irq0();
    }
}
```

A função sigsuspend mantém o kernel aguardando sinais, sem consumir CPU em espera ativa. Quando uma interrupção é recebida, o kernel identifica o evento e executa a rotina de tratamento correspondente.

Exemplo da saída

```
kernel pid=1234 shmid=5678
kernel: A1 pid=1235 criado (READY, MAX=8, roteiro=3R,6W)
kernel: A2 pid=1236 criado (READY, MAX=5, roteiro=-)
kernel: A3 pid=1237 criado (READY, MAX=5, roteiro=2W)
kernel: intercontroller pid=1238

t=  0.0  INICIO  | RUN=A1  | READY=[A2  A3]  | D1=[]  | PC:  A1=0/8  A2=0/5
A3=0/5

[A1] PC=1/8
t=  1.0  IRQ0    | RUN=A2  | READY=[A3  A1]  | D1=[]  | PC:  A1=1/8  A2=0/5
A3=0/5

[A2] PC=1/5
t=  2.0  IRQ0    | RUN=A3  | READY=[A1  A2]  | D1=[]  | PC:  A1=1/8  A2=1/5
A3=0/5

[A3] PC=1/5
t=  3.0  IRQ0    | RUN=A1  | READY=[A2  A3]  | D1=[]  | PC:  A1=1/8  A2=1/5
A3=1/5

[A1] PC=2/8
t=  4.0  IRQ0    | RUN=A2  | READY=[A3  A1]  | D1=[]  | PC:  A1=2/8  A2=1/5
A3=1/5
...
```

4. Explicação do funcionamento

O sistema utiliza um escalonador circular preemptivo, no qual o kernel é dirigido a interrupções. Como o simulador envolve processos concorrentes e eventos assíncronos, foram adotadas algumas decisões de implementação para preservar o comportamento esperado.

4.1. Fatias de tempo e medição de execução

O enunciado propõe a utilização de sleep(1) para simular o tempo de execução das aplicações. Entretanto, durante o desenvolvimento, essa abordagem foi substituída para garantir uma contagem mais consistente das iterações. Nos testes iniciais, o uso direto de sleep(1) produzia comportamentos irregulares: algumas fatias de tempo terminavam sem que a aplicação completasse uma iteração, enquanto outras permitiam completar duas iterações. Isso acontecia porque o tempo de espera continuava sendo contabilizado mesmo quando o processo era suspenso por SIGSTOP. Assim, uma aplicação interrompida no meio de um sleep(1) poderia retomar a execução com parte ou todo o intervalo já transcorrido.

Para corrigir esse problema, a execução de uma iteração passou a ser dividida em 100 intervalos de 10 milissegundos, utilizando nanosleep. Essa mudança em relação ao enunciado foi intencional e busca preservar o tempo efetivo de execução da aplicação, evitando que o período em que ela permanece suspensa seja contabilizado como processamento.

4.2. Troca de contexto via memória compartilhada (SHM)

A troca de contexto utiliza memória compartilhada (shmget/shmat), que é o mecanismo de IPC adotado no trabalho junto com os sinais Unix. Foi alocado um segmento de memória compartilhada contendo um vetor de estruturas AppSlot, uma para cada aplicação. Cada aplicação atualiza seu próprio contador PC nesse segmento. Quando um processo é suspenso por SIGSTOP, o kernel consulta seu PC na memória compartilhada e o armazena no Bloco de Controle de Processo (PCB). Antes de retomar a aplicação com SIGCONT, o kernel restaura o valor do PC no respectivo slot. Dessa forma, o simulador representa a operação de salvar e restaurar o contexto de execução de um processo.

4.3. Mecanismo de syscall e proteção contra avanço

As aplicações solicitam operações de E/S ao kernel por meio de sinais de tempo real:

- SIGRTMIN: leitura (R).
- SIGRTMIN+1: escrita (W).

Os sinais de tempo real são utilizados porque podem ser enfileirados, evitando a perda de notificações quando múltiplas chamadas ocorrem em um intervalo curto. Para impedir que uma aplicação continue executando instruções entre o envio da syscall e a recepção do SIGSTOP pelo kernel, a aplicação ativa uma flag de bloqueio em seu slot de memória compartilhada e permanece em um laço de espera até ser liberada. Assim, o avanço do PC fica interrompido enquanto a solicitação de E/S está pendente.

4.4. Gerenciamento de E/S no InterController

O InterController não utiliza diretamente sleep() para controlar as interrupções. Isso porque um sinal recebido durante o sono poderia interromper a espera e prejudicar a regularidade do relógio. Em vez disso, utiliza referências de tempo absoluto (prox_irq0 e t_fim) e aguarda eventos por meio de sigtimedwait. Essa mudança em relação ao enunciado foi intencional. Os pedidos ao dispositivo D1 são processados estritamente em série. Cada operação leva 3 segundos e, quando o dispositivo está ocupado, as solicitações seguintes permanecem aguardando. Ao concluir uma operação, o InterController envia IRQ1 ao kernel, informando que o primeiro pedido da fila foi atendido.

4.5. Fila D1 e prioridade

Quando uma operação de E/S termina, o InterController envia IRQ1 ao kernel. O kernel remove o primeiro processo da fila FIFO de bloqueados (D1), desativa sua flag de bloqueio na memória compartilhada e o insere na frente da fila de prontos (READY). Essa política implementa a prioridade exigida no enunciado: processos que retornam de E/S passam à frente daqueles que foram retirados da CPU por término do time slice, pois estes são inseridos no final da fila.

Se a CPU estiver ociosa porque todos os processos vivos estão bloqueados, o processo é escalonado imediatamente. Caso outro processo esteja executando, ele permanece na CPU até o fim de sua fatia de tempo. No IRQ0 seguinte, o escalonador seleciona o processo que retornou da E/S. Quando IRQ0 e IRQ1 estão pendentes no mesmo ciclo de tratamento, o kernel processa primeiro IRQ1. Assim, a aplicação liberada da E/S entra na frente da fila antes de ocorrer o próximo escalonamento. O parâmetro da syscall também faz parte do contexto. No momento da syscall, o kernel guarda a operação pedida (R ou W) no PCB do processo, junto com o PC. Ela aparece no log na fila de D1 (por exemplo, D1=[A3:W A1:R]) e é apagada do PCB quando o IRQ1 confirma o fim daquela E/S.

5. Testes

Os testes executam o próprio simulador com configurações fixas, verificando seu comportamento em diferentes situações de escalonamento e operações de E/S.

Para executar um teste:

```
testes/run_teste.sh N
```

O valor de N varia de 1 a 4.

O script compila o projeto, executa o kernel e, após todas as aplicações terminarem, envia SIGINT, equivalente a pressionar Ctrl+C.

São realizadas quatro verificações:

1. A linha do tempo produzida corresponde à esperada em testes/esperado/testeN.txt.
2. Todas as aplicações terminam com PC = MAX.

3. Nenhum processo do teste (kernel, InterController e aplicações) permanece ativo após o encerramento.
4. O segmento de memória compartilhada é removido.

O log completo de cada execução é armazenado em testes/saida/testeN.log. Cada linha mostra o tempo, o evento (INICIO, IRQ0, IRQ1, SYS ou FIM), o processo em execução (RUN, com -- para CPU ociosa), a fila de prontos (READY), a fila de D1 com a operação pedida e o PC/MAX de cada processo. (F) indica que o processo terminou.

Teste 1 – Bloqueio e retorno prioritário

```
./kernel 6:2R 6 6
```

Em t=3, A1 solicita leitura (R) e é bloqueado imediatamente. Enquanto aguarda, A2 e A3 se alternam na CPU. Em t=6, a E/S de A1 termina e ele é inserido na frente da fila de prontos (READY=[A1 A2]). Por isso, A1 executa em t=7, antes de A2, que havia sido retirado da CPU por término de fatia de tempo.

Teste 2 – Múltiplas operações de E/S

```
./kernel 8:3R,6W 5 5:2W
```

Em t=5, A3 solicita escrita (W) e A1 solicita leitura (R) logo em seguida, formando a fila: D1=[A3:W A1:R]. Como as operações de E/S são processadas em série, os IRQ1 ocorrem em t=8 e t=11, liberando A3 e depois A1, com 3 segundos de intervalo. Entre t=7 e t=8, a CPU permanece ociosa (RUN=--). Quando a E/S termina, o processo liberado é escalonado imediatamente.

Teste 3 – Todos os processos bloqueados

```
./kernel 4:2R 4:1W 4:1R
```

Os três processos solicitam operações de E/S em t=1, deixando a CPU ociosa até t=4. Os IRQ1 ocorrem em t=4, t=7 e t=10, liberando as aplicações na ordem em que solicitaram as operações: A2, A3 e A1. Esse teste verifica o comportamento do escalonador quando não há processos prontos e o atendimento FIFO do dispositivo D1.

Teste 4 – Seis processos e término em tempos diferentes

```
./kernel 4 4:1W 4:1R 5 3 4
```

O teste utiliza seis aplicações, com valores distintos de MAX. A2 e A3 solicitam E/S em t=1. Quando retornam, são inseridos na frente da fila de prontos, mesmo com outros processos aguardando. A2 retorna em t=4 e A3 em t=7, momento em que a fila fica: READY=[A3 A6 A1 A2 A4]. As aplicações terminam em momentos diferentes, entre t=13 e t=16, permitindo verificar o gerenciamento de múltiplos processos e a atualização das filas durante a execução.

6. Resultados

Os quatro testes passaram nas quatro verificações realizadas, confirmando o término correto das aplicações, o comportamento esperado das filas, a finalização dos processos filhos e a remoção da memória compartilhada.

As syscalls dos testes começam em t ≥ 1. Quando uma syscall ocorre em t=0, a conclusão da E/S pode coincidir aproximadamente com uma interrupção de relógio, tornando a ordem de chegada dos sinais dependente do escalonamento real do sistema

operacional hospedeiro. Por esse motivo, os testes utilizam configurações que evitam essa condição de concorrência temporal, permitindo comparar os resultados com linhas do tempo determinísticas.